Reference Architectures

We show that our methodology can derive well-known reference architectures for data-intensive systems. For each reference architecture, we:

- Model the requirements it addresses as an SDL scenario;
- Describe the architecture itself in our ADL;
- Prove that the output of our methodology for that scenario is consistent with the expected architecture.

1 Data Lake

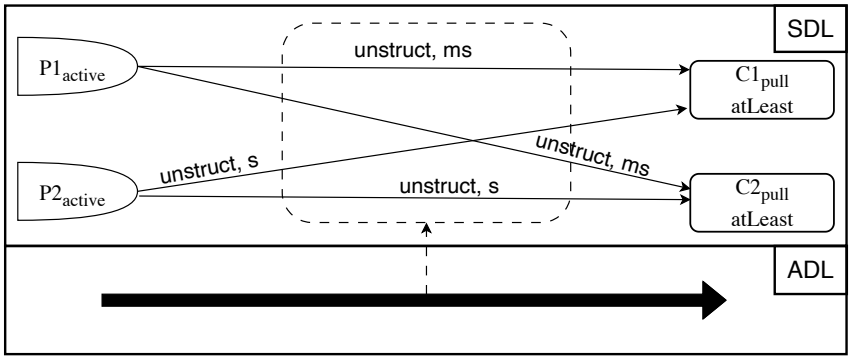


Figure 1: Data lake: SDL scenario (top) and ADL architecture (bottom).

Scenario Description Language. The data lake architecture addresses the need to ingest and store raw data from heterogeneous sources, making it available for later consumption on demand without any intermediate transformation. We model this as a scenario with no processing nodes, where data flows directly from producers to consumers.

In a data lake, sources continuously push data into the system (e.g., application logs, IoT sensors, event streams) at different rates. We represent this using two active producers ($\mathbf{P1_{active}}$ and $\mathbf{P2_{active}}$) emitting unstructured data

at different frequencies: P1 at a sub-millisecond period (0.001s) and P2 at a second period (1s).

Downstream applications query the stored data on demand, and require no data loss during ingestion, tolerating duplicates, which can be resolved at query time. We represent this using two pull consumers ($\mathbf{C1_{pull}}$ and $\mathbf{C2_{pull}}$), each with an *at-least-once* delivery guarantee.

We use two producers and consumers for simplicity of presentation. Without loss of generality, the scenario can be extended to an arbitrary number of producers and consumers. Every producer is connected to every consumer, reflecting the principle that all ingested data is made available to all consumers.

Architecture Description Language. The data lake architecture consists exclusively of persistent links connecting producers directly to consumers, with no processing components in between. Each persistent link maps to a durable storage system (e.g., HDFS, AWS S3) that ingests raw data from sources and makes it available for on-demand querying. The absence of processing components reflects the core principle of the data lake: data is stored as-is and any transformation is deferred to query time.

Methodology Output and Comparison. Our methodology produces exactly one candidate architecture, with all links set to *persistent*. The result is fully consistent with the data lake ADL described above.

2 Lambda

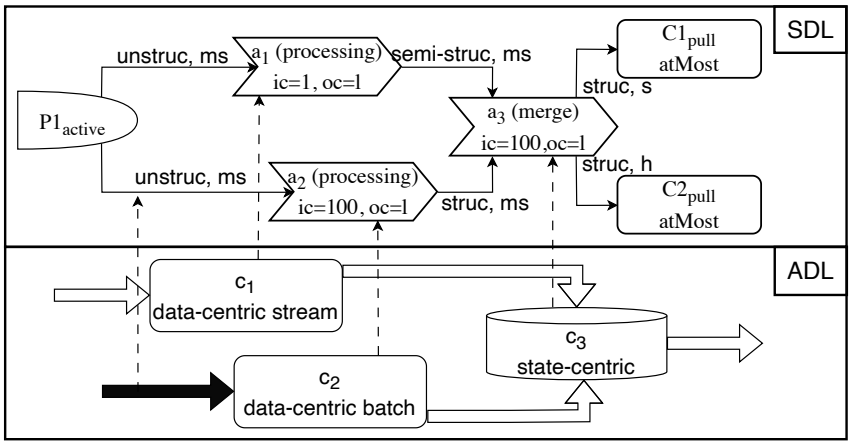


Figure 2: Data lake: SDL scenario (top) and ADL architecture (bottom).

Scenario Description Language. The Lambda architecture addresses the need to perform both lightweight, low-latency processing and heavy computations on large volumes of data (e.g., historical data), merging their results to

serve consumers with different requirements. We model this as a scenario with two parallel processing paths that converge into a merge node before reaching the consumers. As with all reference architectures presented in this section, the specific data types and parameters represent one possible instantiation. Concrete systems may adapt them depending on the application domain.

A single active producer ($\mathbf{P1_{active}}$) continuously pushes unstructured data at a sub-millisecond period (0.001s), representing a high-throughput event source.

Two processing nodes operate in parallel on the producer's data. Node **a1** performs lightweight transformations with a 1-to-1 input-to-output cardinality, producing semi-structured data at the same rate as the producer (0.001s). Node **a2** performs heavy computations over large volumes of data, modeled by a 100-to-1 input-to-output cardinality, producing structured results at an hourly period (3600s). A merge node **a3** integrates results from both paths, with a 100-to-1 input-to-output cardinality and structured output.

Two pull consumers query the merged results at different frequencies: $\mathbf{C1_{pull}}$ at a second period (1s) and $\mathbf{C2_{pull}}$ at an hourly period (3600s), both with an *at-most* delivery guarantee. Consumers are *pull* because they query the serving layer on demand. The *at-most* guarantee reflects a design that prioritizes low latency over completeness: occasional data loss is acceptable because the batch layer periodically recomputes accurate results.

Architecture Description Language. The Lambda architecture is modeled using three components. A stream processing component (**c1**) implements the speed layer, performing lightweight, continuous transformations on incoming data. A batch processing component (**c2**) implements the batch layer, periodically recomputing results over large volumes of data. A state-centric component (**c3**) implements the serving layer, storing the merged results and making them available for on-demand queries. The only persistent link in the architecture is the one entering **c2**: it accumulates incoming data to provide the batch component with its complete input dataset. All other links are volatile, as the stream component processes data in-flight and the state-centric component is directly queryable by consumers.

Methodology Output and Comparison. The methodology decomposes the scenario into two data flows, one per consumer. In both data flows, **a2** is unambiguously assigned to a batch processing component. In contrast, **a1** and **a3** are sensitive to cost assumptions. The sensitivity analysis produces multiple candidate architectures. Among them, one with a support score of 3.7% assigns **a1** to stream processing, **a2** to batch processing, and **a3** to state-centric, with the only persistent link being the ingestion link into the batch component. This candidate is structurally identical to the Lambda ADL described above. While it is not the architecture with the highest support score, its presence among the candidates confirms that the methodology correctly identifies the Lambda pattern as a viable design. The three highest-scoring candidates differ from the Lambda in two aspects. First, they all assign n3 to a stream processing component instead of a state-centric one. As a consequence, the links to both pull consumers become persistent, since there is no queryable state upstream. Sec-

ond, they vary in their implementation of n1: one uses stream processing alone, another adds a state-centric component alongside stream, and a third replaces stream with batch plus a state-centric component, adding a further persistent ingestion link. Overall, these candidates favor stream-based merging with persistent output links, resulting in architectures with more persistent storage than the Lambda. Notably, the conditions under which the Lambda emerges as the preferred candidate reflect common practical assumptions: stream processing is the natural and most cost-effective choice for lightweight, stateless transformations such as those performed by **a1**, and a state-centric component is cost-effective for **a3** whenever the retrieval cost rc is not prohibitively high. Under these widely applicable assumptions, the Lambda is the top-ranked candidate, confirming that our methodology identifies it as the expected architecture for this class of scenarios.

3 Kappa

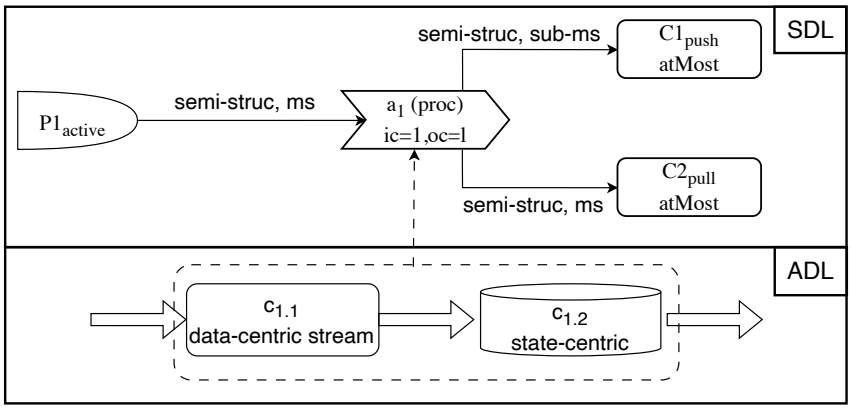


Figure 3: Data lake: SDL scenario (top) and ADL architecture (bottom).

Scenario Description Language. The Kappa architecture addresses scenarios where all processing can be handled through a single stream processing path, avoiding the complexity of maintaining separate batch and speed layers. This is possible when the volume of data considered at each computation is limited (e.g., by restricting the temporal range under analysis), keeping the computational cost manageable for stream processing. We model this as a scenario with a single processing node serving two consumers.

A single active producer ($\mathbf{P1_{active}}$) continuously pushes semi-structured data at a period of 0.1s. A single processing node **a1** performs lightweight transformations with a 1-to-1 input-to-output cardinality. The combination of a moderate input rate and a low-cardinality action models the defining assump-

tion of the Kappa architecture: the processing workload is simple enough that a single stream path can handle it without resorting to a separate batch layer.

$\mathbf{C1_{push}}$ receives updates at a period of 0.0001s, faster than the producer rate, modeling a fan-out pattern in which each processed event triggers multiple downstream notifications (e.g., a real-time alerting system), a scenario that naturally calls for a Kappa architecture, as it requires continuous, low-latency processing without the need to recompute over historical data. $\mathbf{C2_{pull}}$ queries results on demand at a period of 0.001s, representing a downstream application that reads the current processed state (e.g., a monitoring dashboard). The mix of push and pull consumers reflects a typical Kappa deployment, where the stream processor both drives reactive downstream applications and maintains a serving layer that can be queried on demand. Both consumers require an *at-most* delivery guarantee. This is characteristic of Kappa deployments, where the system relies on the continuous arrival of fresh events to keep consumers up to date, making occasional losses acceptable.

Architecture Description Language. The Kappa architecture is modeled using two components. A stream processing component (**c1**) performs continuous, lightweight transformations on incoming data, implementing the single processing path that characterizes the Kappa design. A state-centric component (**c2**) stores the processed results and makes them available for downstream consumption. All links in the architecture are volatile.

Methodology Output and Comparison. The methodology produces two candidate architectures. The highest-scoring candidate (support 66.7%) assigns **a1** to a stream processing component, with a persistent link toward $\mathbf{C2_{pull}}$. This candidate differs from the Kappa ADL as it lacks the state-centric serving layer, and it compensates with a persistent link to provide the pull consumer with a durable, queryable source. The second candidate (support 33.3%) assigns **a1** to a stream processing component and adds a state-centric component for serving, with all links volatile, which is structurally identical to the Kappa ADL described above.

Notably, the conditions under which the Kappa emerges reflect common practical assumptions: stream processing is the natural choice for lightweight 1-to-1 transformations, and a state-centric component is cost-effective for serving pull consumers whenever the retrieval cost rc is not prohibitively high. Under this assumption, the methodology correctly identifies the Kappa as the optimal architecture for this class of scenarios.

4 Liquid

Scenario Description Language. The Liquid architecture addresses scenarios where continuous, real-time processing must be combined with guaranteed data delivery. Unlike the Kappa architecture, which tolerates data loss in favor of low latency, Liquid ensures that no data item is lost during ingestion or processing, making it suitable for applications where completeness is critical

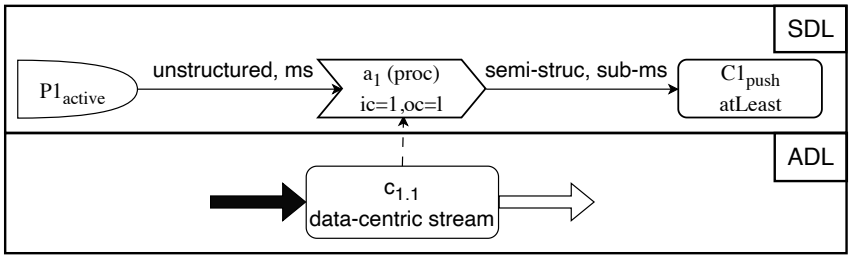


Figure 4: Data lake: SDL scenario (top) and ADL architecture (bottom).

(e.g., event-driven monitoring or reliable alerting pipelines). We model this as a scenario with a single processing node connecting one active producer to one push consumer.

A single active producer ($\mathbf{P1_{active}}$) continuously pushes unstructured data at a period of 0.1s (e.g., sensor readings or event logs). A single processing node **a1** performs lightweight transformations with a 1-to-1 input-to-output cardinality, producing structured output.

The consumer $\mathbf{C1_{push}}$ receives processed data at a much higher rate (period 0.0001s), representing a downstream application that requires enriched events at high throughput (e.g., a real-time alerting system or monitoring pipeline that fans out results to multiple subscribers). It requires an *at-least-once* delivery guarantee, ensuring that every data item produced by the source is reliably processed and delivered even in the presence of failures. This is the key requirement distinguishing the Liquid architecture from the Kappa: the system must not drop data, and must be able to recover and replay it if needed.

Architecture Description Language. The Liquid architecture is modeled using a single stream processing component (**c1**) with a persistent incoming link. The persistent link from $\mathbf{P1_{active}}$ to **c1** is provides a durable buffer that stores data upon arrival, enabling replay in case of failure and thus satisfying the consumer's at-least-once delivery requirement. The stream processing component **c1** continuously transforms incoming data in real time, producing structured output delivered directly to $\mathbf{C1_{push}}$ via a volatile link. No persistent output link is required, as the push consumer receives data continuously and does not issue on-demand queries.

Methodology Output and Comparison. The methodology decomposes the scenario into a single data flow for the unique consumer $\mathbf{C1_{push}}$, and produces exactly one candidate architecture: a stream processing component with a persistent input link and a volatile output link, which is structurally identical to the Liquid ADL described above. This result confirms that the Liquid architecture emerges naturally from our methodology.